

PROJECT REPORT OF CO-OP Project at Industry (Module-2)
ON
Security Assurance and Compliance Verification in AI-Generated Software Systems
submitted in partial fulfilment of the requirements for the award of degree of
BACHELOR OF ENGINEERING
In
COMPUTER SCIENCE AND ENGINEERING

Submitted by:

Abhayjeet Singh (2210990031),

Simran Jaswal (2210992383),

Abhishek Chaudhary (2210991161),

Chirag Mehta (2210991463)

Supervised By:

Dr. Preeti Saini

Assistant Professor

Chitkara University, Punjab



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
CHITKARA
UNIVERSITY INSTITUTE OF ENGINEERING AND TECHNOLOGY
CHITKARA UNIVERSITY, PUNJAB, INDIA

CONTENTS

S.No.	Title	Page No.
1	Acknowledgement	4
2	Abstract	5
3	Introduction	6
4	Methodology	7-8
5	Tools and Technologies	9-10
6	Implementation	11-12
7	Major Findings/Outcomes/Output/Results	13-14
8	Conclusion and Future Scope	15
9	References	16

DECLARATION

I hereby certify that the work which is being presented in the project report entitled “Security Assurance and Compliance Verification in AI-Generated Software Systems” in partial fulfilment of requirement for the award of the degree of Bachelor of Engineering (Computer Science and Engineering) submitted in the department of Computer Science and Engineering at Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab, India, is an authentic record of my own work carried out under the supervision of **Dr. Preeti Saini**. The matter presented in this project report has not been submitted in any other university/institute for the award of any degree.

Place: Rajpura

Abhayjeet Singh, Simran Jaswal,

Abhishek Chaudhary, Chirag Mehta

Date: April 28, 2026

2210990031, 2210992383,

2210991161, 2210991463

This is to certify that the above statement made by the candidate is correct to the best of my knowledge and belief.

Dr. Preeti Saini

Assistant Professor

Department of Computer Science and Engineering

Chitkara University Institute of Engineering and Technology,

Chitkara University, Punjab, India

1. ACKNOWLEDGEMENT

We would like to express our sincere gratitude to all those who supported us throughout the completion of our project titled **“Security Assurance and Compliance Verification in AI-Generated Software Systems”**

First and foremost, we would like to thank our project mentor **Dr. Preeti Saini**, for providing valuable guidance, continuous encouragement, and constructive suggestions during the development of this project. Their expert advice and constant motivation helped us in understanding the practical aspects of the project and completing it successfully.

We are also thankful to the Department of Computer Science and Engineering, Chitkara University Institute of Engineering and Technology, for providing us with the opportunity and resources required to work on this project as part of our academic curriculum.

We would like to extend our gratitude to all faculty members and our peers who supported us with their suggestions and feedback during the course of this work.

This project has been a valuable learning experience for us, enhancing our technical and analytical skills while providing practical exposure to **AI-generated software systems, security assurance methodologies, vulnerability assessment, compliance verification, and secure software development practices.**

2. ABSTRACT

The rapid adoption of large language model (LLM)-powered coding assistants in modern software development has introduced significant security concerns alongside productivity benefits. While AI-generated code can accelerate development and automate repetitive programming tasks, it often produces insecure implementations that may pass traditional functional testing while still containing hidden vulnerabilities. Conventional quality assurance workflows are therefore insufficient for identifying many security risks introduced by AI-assisted coding practices. To address this challenge, this project presents a comprehensive security evaluation framework for analyzing vulnerabilities in AI-generated code using automated and manual assessment techniques.

The proposed framework integrates Static Application Security Testing (SAST) tools with manual code review to systematically evaluate code artifacts generated by large language models. Vulnerabilities are identified, categorized, and analyzed according to the OWASP Top 10 (2021) security standards. The study focuses on detecting critical weaknesses such as injection flaws, broken access control, cryptographic failures, and insecure coding patterns commonly observed in AI-generated software components. A structured multi-stage vulnerability detection pipeline is implemented to improve the accuracy and reliability of security assessment results.

Experimental analysis reveals recurring vulnerability patterns in AI-generated code, demonstrating that functional correctness does not necessarily imply security compliance. The findings highlight the limitations of traditional verification workflows when applied to AI-assisted development environments and emphasize the importance of integrating dedicated security validation mechanisms into the software development lifecycle. This project demonstrates that combining automated security analysis with expert manual review can significantly enhance the detection of vulnerabilities in AI-generated code, thereby improving software reliability, strengthening secure development practices, and reducing the risk of deploying insecure applications in real-world systems.

3. Introduction

The rapid advancement of large language models (LLMs) has transformed modern software development by introducing AI-powered coding assistants capable of generating functional code from natural language prompts. Tools such as GitHub Copilot, Amazon CodeWhisperer, and Anthropic Claude are increasingly being integrated into professional development workflows, enabling developers to automate repetitive tasks, accelerate software production, and improve coding efficiency. These systems can generate complete functions, authentication modules, database interactions, and application logic with minimal human intervention. As a result, AI-assisted programming has become an influential component of contemporary software engineering practices, with growing adoption across both industry and academia.

Despite the significant productivity benefits offered by AI-generated code, concerns regarding software security have emerged as a critical challenge. Most large language models are primarily optimized for functional correctness, meaning the generated code is designed to compile successfully, execute properly, and satisfy user requirements or unit tests. However, functional correctness does not guarantee security compliance. AI-generated code may contain hidden vulnerabilities that are not detected through traditional testing and quality assurance processes. Consequently, software systems developed using AI-assisted coding tools may unknowingly introduce security weaknesses into production environments, thereby increasing the risk of exploitation and cyberattacks.

The security risks associated with AI-generated code are particularly concerning because vulnerabilities can be reproduced at scale across multiple applications and codebases. If insecure coding patterns are present in the training data or inadequately represented in secure coding standards, large language models may repeatedly generate the same vulnerable implementations for different users and projects. Therefore, there is a growing need for systematic evaluation methodologies capable of identifying and analyzing vulnerabilities in AI-generated code before deployment.

To address these concerns, this study adopts the OWASP Top 10 (2021) framework as the primary reference standard for vulnerability classification and analysis. The OWASP Top 10 provides a widely recognized taxonomy of the most critical web application security risks, including injection flaws, broken access control, cryptographic failures, and insecure design practices. In particular, the inclusion of “Insecure Design” in the 2021 edition highlights the importance of evaluating architectural and design-level security risks.

This paper presents a controlled experimental study on the security assessment of AI-generated code using a layered vulnerability detection framework. The proposed approach combines Static Application Security Testing (SAST) tools with expert manual code review to identify, categorize, and analyze vulnerabilities in code artifacts generated by large language models. The study further develops a structured vulnerability dataset mapped to OWASP Top 10 categories, severity levels, and remediation outcomes. Through this research, the paper aims to provide practical insights into the security limitations of AI-assisted software development and proposes recommendations for integrating effective security validation mechanisms within AI-augmented development workflows.

4. Methodology

The AI-Generated Code Security Evaluation Framework follows a structured methodology to analyze the security vulnerabilities present in code generated by large language model (LLM)-based coding assistants. The methodology includes prompt generation, code collection, automated and manual vulnerability analysis, and result evaluation. The complete process is divided into the following stages:

4.1 Research Design

The study follows a controlled experimental design in which the AI code generation tool acts as the independent variable, while the number, type, and severity of vulnerabilities identified in generated code serve as the dependent variables. A within-subjects approach is used so that identical prompts are submitted to all AI tools, ensuring fair comparison and minimizing variations caused by task complexity.

4.2 Prompt Corpus Construction

A dataset of 30 programming task prompts is created to represent common real-world web application development scenarios. The prompts are divided into three major categories:

- User Authentication
- Data Management
- Data Transmission

The prompts include functionalities such as login systems, password reset mechanisms, database operations, REST API creation, file upload handling, and inter-service communication. All prompts are written in natural language without explicitly mentioning security requirements to simulate realistic developer interactions with AI coding assistants.

4.3 Code Generation and Collection

The prepared prompts are submitted to three AI-powered coding assistants:

- OpenAI GPT-4o
- GitHub Copilot
- Anthropic Claude 3.5 Sonnet

Each prompt is executed independently in a fresh session to avoid contextual influence. Generated outputs are collected without any manual modification and stored in a structured dataset along with metadata such as tool name, prompt identifier, timestamp, and programming language. A total of 90 Python code artifacts are collected, with 30 artifacts generated from each AI tool.

4.4 Automated Static Vulnerability Analysis

The generated code artifacts are analyzed using Static Application Security Testing (SAST) tools to identify potential vulnerabilities. The framework uses:

- Bandit v1.7.5 for Python-specific security analysis
- Semgrep v1.45 with OWASP Top 10 rule sets

These tools detect security weaknesses such as injection vulnerabilities, insecure configurations, weak cryptographic practices, and broken access control issues. The findings are categorized based on CWE identifiers, OWASP Top 10 categories, severity levels, and confidence scores.

4.5 Dependency and Configuration Analysis

Artifacts that use external libraries are further analyzed using pip-audit to identify known vulnerabilities in third-party dependencies. Configuration-related security issues such as enabled debug mode, insecure CORS settings, and disabled SSL verification are detected using custom security rules developed specifically for the framework.

4.6 Manual Expert Code Review

A selected subset of generated code artifacts undergoes manual security review by experts trained in application security. The review process follows guidelines adapted from the OWASP Code Review Guide and evaluates factors such as:

- Input validation
- Output encoding
- Authentication and session management
- Access control
- Error handling
- Logging and monitoring

4.7 Vulnerability Classification and Analysis

All identified vulnerabilities are consolidated and mapped to the OWASP Top 10 (2021) categories. The vulnerabilities are further classified according to severity levels such as LOW, MEDIUM, HIGH, and CRITICAL. Duplicate findings from multiple tools are removed to create a refined vulnerability dataset for analysis.

4.8 Result Evaluation

The collected results are analyzed to identify recurring vulnerability patterns across different AI coding assistants. Comparative analysis is performed to evaluate the security quality of generated code, the effectiveness of automated security tools, and the frequency of different vulnerability categories. Visual representations such as vulnerability distribution graphs, severity comparison charts, and OWASP category analysis are used to interpret the findings and evaluate the overall security posture of AI-generated code.

5. Tools and Technologies

The development of the AI-Generated Code Security Evaluation Framework involved the use of several tools and technologies for AI-based code generation, vulnerability detection, dependency analysis, data processing, and result visualization.

5.1 Python

Python was used as the primary programming language for implementing the experimental framework, automation scripts, and security analysis pipeline. It provided extensive support for API integration, data processing, static analysis tools, and machine learning-based evaluation.

5.2 GPT-4o

OpenAI GPT-4o was used as one of the AI-powered code generation systems in the study. It generated Python code artifacts for different web application development tasks, enabling security evaluation of AI-generated outputs.

5.3 GitHub Copilot

GitHub Copilot was used as an AI coding assistant for generating source code within the Visual Studio Code environment. It assisted in producing code completions and function implementations based on the experimental prompts.

5.4 Claude 3.5 Sonnet

Anthropic Claude 3.5 Sonnet was used as an additional large language model for code generation. The tool was evaluated to compare the security quality and vulnerability patterns present in its generated code outputs.

5.5 Bandit

Bandit v1.7.5 was used as a Static Application Security Testing (SAST) tool for identifying security vulnerabilities in Python code. It helped detect issues such as insecure function usage, weak cryptographic practices, injection risks, and unsafe configurations.

5.6 Semgrep

Semgrep v1.45 was used for rule-based static security analysis using the OWASP Top 10 community ruleset. It enabled detection of vulnerabilities related to injection flaws, broken access control, insecure configurations, and other common security weaknesses.

5.7 pip-audit

pip-audit was used for dependency vulnerability analysis. It scanned third-party Python packages used in generated artifacts and identified known Common Vulnerabilities and Exposures (CVEs) using the Python Packaging Advisory Database (PyPA).

5.8 Pandas

Pandas was used for organizing, processing, and analyzing vulnerability data collected from different security analysis tools. It helped in creating structured datasets containing vulnerability categories, severity levels, CWE mappings, and detection statistics.

5.9 Scikit-learn

Scikit-learn was used for statistical analysis and evaluation tasks related to vulnerability classification and inter-review agreement analysis. It supported data preprocessing and analytical operations performed during the experimental study.

5.10 Matplotlib

Matplotlib was used to visualize the experimental results through graphs and charts such as vulnerability distribution, severity comparison, OWASP category analysis, and tool-wise vulnerability detection statistics. These visualizations helped in interpreting the findings effectively.

5.11 Jupyter Notebook / Visual Studio Code

The implementation, testing, and analysis of the framework were carried out using Jupyter Notebook and Visual Studio Code. These environments provided support for code execution, debugging, API integration, security testing, and result visualization.

6. Implementation

The implementation of the AI-Generated Code Security Evaluation Framework was carried out in multiple stages, including prompt preparation, AI-based code generation, automated vulnerability analysis, dependency scanning, manual security review, and result visualization.

6.1 Prompt Dataset Preparation

A structured dataset of 30 programming task prompts was created to simulate real-world web application development scenarios. The prompts covered functionalities related to user authentication, data management, and data transmission. Each prompt was written in natural language without explicitly mentioning security requirements in order to replicate practical developer interactions with AI coding assistants.

6.2 AI-Based Code Generation

The prepared prompts were submitted to multiple AI-powered coding assistants, including OpenAI GPT-4o, GitHub Copilot, and Anthropic Claude 3.5 Sonnet. Each prompt was executed independently in isolated sessions to avoid contextual influence from previous interactions. The generated Python code artifacts were collected and stored systematically without any manual modification.

6.3 Automated Static Security Analysis

The generated code artifacts were analyzed using Static Application Security Testing (SAST) tools. Bandit and Semgrep were integrated into the framework to automatically detect vulnerabilities such as:

- Injection flaws
- Broken access control
- Insecure configurations
- Weak cryptographic implementations
- Unsafe function usage

The analysis results included vulnerability identifiers, severity levels, CWE mappings, and OWASP Top 10 classifications.

6.4 Dependency and Configuration Scanning

Artifacts containing external package imports were scanned using pip-audit to identify known vulnerabilities in third-party dependencies. In addition, custom Semgrep security rules were implemented to detect configuration-level issues such as:

- Enabled debug mode
- Insecure CORS policies
- Disabled SSL certificate verification
- Hardcoded sensitive information

This stage improved the overall security assessment coverage beyond source code analysis.

6.5 Manual Security Review

A subset of generated artifacts was selected for expert manual review to validate the automated findings and identify vulnerabilities that static analysis tools might miss. The review process followed guidelines based on the OWASP Code Review Guide and focused on areas such as:

- Input validation
- Authentication and session management
- Access control mechanisms
- Error handling
- Secure data storage
- Logging and monitoring

The review process also helped measure the accuracy and false positive rates of automated detection tools.

6.6 Vulnerability Classification and Dataset Creation

All identified vulnerabilities were consolidated into a structured dataset. Duplicate findings from multiple tools were removed, and each vulnerability was categorized according to the OWASP Top 10 (2021) framework and severity levels including LOW, MEDIUM, HIGH, and CRITICAL. The final dataset was used for comparative analysis of vulnerability patterns across different AI code generation tools.

6.7 Result Visualization and Analysis

The experimental results were visualized using graphs and charts generated with Matplotlib, including:

- Vulnerability Distribution by OWASP Category
- Severity Level Comparison
- Tool-wise Vulnerability Analysis
- False Positive Rate Comparison
- Vulnerability Frequency Distribution

These visualizations helped evaluate the security posture of AI-generated code and analyze the effectiveness of different vulnerability detection techniques.

7. Major Findings/Outcomes/Output/Results

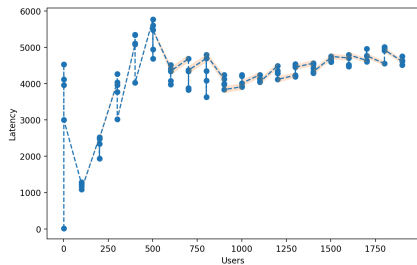
The experimental results obtained from the AI-Assisted Load Testing and Failure Prediction Framework demonstrated the effectiveness of combining load testing with machine learning for predictive performance analysis.

7.1 System Performance Under Load

During load testing, the system performed efficiently under low and moderate user loads, maintaining low latency and minimal error rates. However, as the number of concurrent users increased, the system showed a gradual rise in response latency and error rate, indicating increased stress on system resources.

This behaviour helped identify the load range within which the system operated reliably.

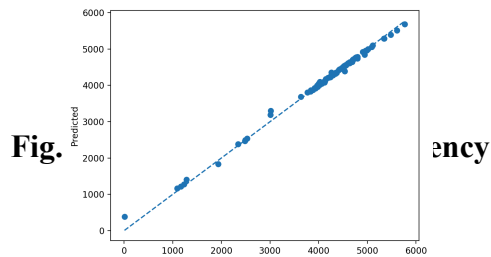
Fig. 1: Latency vs Number of Concurrent Users



7.2 Latency Prediction Accuracy

The machine learning models were able to predict system latency with good accuracy based on the collected performance metrics. Among the regression models used, the Voting Regressor produced the most accurate results by combining the predictions of multiple models.

The comparison between actual latency and predicted latency showed a close match, validating the effectiveness of the predictive model.



7.3 Failure Threshold Estimation

The Logistic Regression model successfully estimated the probability of system failure as the load increased. The results showed that the framework could identify the approximate maximum number of concurrent users the system could handle before performance degradation became critical.

This prediction helps in planning system capacity and avoiding unexpected failures.

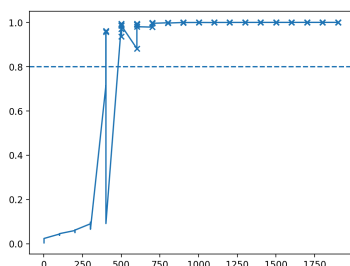
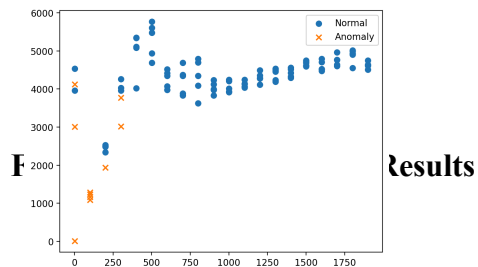


Fig. 3: Failure Probability vs Number of Users

7.4 Anomaly Detection

The Isolation Forest algorithm effectively detected abnormal system behaviour such as sudden spikes in latency and unusual performance patterns. This enabled early identification of performance bottlenecks and potential failures.

Anomaly detection improved the reliability of the framework by providing proactive alerts before complete system breakdown.



7.5 Digital Twin Simulation Results

The digital twin simulation successfully modeled system behaviour under varying loads using the trained machine learning models. This allowed prediction of system performance without applying high stress on the actual environment.

The simulation results were useful for evaluating future load scenarios and estimating performance limits safely.

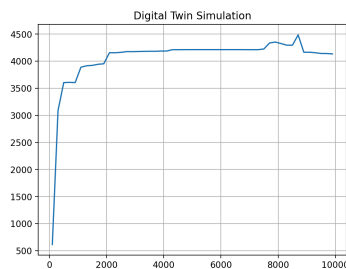


Fig. 5: Digital Twin Simulation

7.6 Overall Outcome

The overall implementation proved that integrating machine learning with load testing significantly improves the traditional performance testing process by enabling:

- Accurate latency prediction
- Failure probability estimation
- Early anomaly detection
- Safe performance simulation using digital twin models

This framework provides developers with proactive insights for optimizing system performance and improving application reliability.

8. Conclusion and Future Scope

The AI-Generated Code Security Evaluation Framework successfully demonstrated how security assessment techniques can be integrated with AI-assisted software development to identify vulnerabilities in code generated by large language models. By combining automated Static Application Security Testing (SAST), dependency analysis, and manual expert review, the framework effectively analyzed the security posture of AI-generated code across multiple web application development scenarios.

The experimental results revealed that although AI coding assistants improve development speed and productivity, they frequently generate code containing security weaknesses such as injection vulnerabilities, broken access control, insecure configurations, and cryptographic failures. The study also showed that many of these vulnerabilities may remain undetected by traditional functional testing approaches, highlighting the importance of dedicated security validation mechanisms in AI-augmented development workflows.

The use of multiple analysis layers improved the accuracy and reliability of vulnerability detection by combining automated scanning with expert manual assessment. Mapping vulnerabilities to the OWASP Top 10 (2021) framework further helped in understanding common security risks present in AI-generated software components. The findings of this project emphasize the need for integrating secure coding practices, automated security gates, and continuous vulnerability assessment into modern AI-assisted software engineering processes.

8.1 Future Scope

The project can be further enhanced in the following ways:

- Extending the framework to support additional programming languages and development frameworks
- Integrating dynamic application security testing (DAST) along with SAST for improved vulnerability coverage
- Applying deep learning-based vulnerability detection models for more advanced security analysis
- Expanding the evaluation to include more AI coding assistants and newer large language models
- Developing a real-time security assessment dashboard for continuous monitoring of AI-generated code
- Integrating automated remediation suggestions and secure code recommendations within the framework

These improvements can make the framework more scalable, intelligent, and effective for securing AI-assisted software development environments in real-world enterprise applications.

9 .References

[1] Flask Documentation, “Flask Web Development Framework”

Available: <https://flask.palletsprojects.com/>

[2] aiohttp Documentation, “Asynchronous HTTP Client/Server for Python”

Available: <https://docs.aiohttp.org/>

[3] asyncio Documentation, “Asynchronous I/O in Python”

Available: <https://docs.python.org/3/library/asyncio.html>

[4] Pandas Documentation, “Data Analysis and Manipulation Tool”

Available: <https://pandas.pydata.org/>

[5] Scikit-learn Documentation, “Machine Learning in Python”

Available: <https://scikit-learn.org/>

[6] Matplotlib Documentation, “Visualization with Python”

Available: <https://matplotlib.org/>

[7] Apache JMeter User Manual, “Performance Testing Tool”

Available: <https://jmeter.apache.org/>